

Database Management Systems

Chapter 4 *Functional Dependency –Contd.*

Decomposition

- Decomposition in DBMS removes redundancy, anomalies and inconsistencies from a database by dividing the table into multiple tables.
- The following are the types –
- Lossless Decomposition
- Decomposition is lossless if it is feasible to reconstruct relation R from decomposed tables using Joins. This is the preferred choice. The information will not lose from the relation when decomposed. The join would result in the same original relation.

Let us see an example –

<EmpInfo>

Emp_ID	Emp_Name	Emp_Age	Emp_Location	Dept_ID	Dept_Name
E001	Jacob	29	Alabama	Dpt1	Operations
E002	Henry	32	Alabama	Dpt2	HR
E003	Tom	22	Texas	Dpt3	Finance

Decompose the above table into two tables:

<EmpDetails>

Emp_ID	Emp_Name	Emp_Age	Emp_Location
E001	Jacob	29	Alabama
E002	Henry	32	Alabama
E003	Tom	22	Texas

<DeptDetails>

Dept_ID	Emp_ID	Dept_Name
Dpt1	E001	Operations
Dpt2	E002	HR
Dpt3	E003	Finance

Now, Natural Join is applied on the above two tables –

The result will be –

Emp_ID	Emp_Name	Emp_Age	Emp_Location	Dept_ID	Dept_Name
E001	Jacob	29	Alabama	Dpt1	Operations
E002	Henry	32	Alabama	Dpt2	HR
E003	Tom	22	Texas	Dpt3	Finance

Therefore, the above relation had lossless decomposition i.e. no loss of information.

Lossy Decomposition

- As the name suggests, when a relation is decomposed into two or more relational schemas, the loss of information is unavoidable when the original relation is retrieved. Let us see an example –

<EmpInfo>

Emp_ID	Emp_Name	Emp_Age	Emp_Location	Dept_ID	Dept_Name
E001	Jacob	29	Alabama	Dpt1	Operations
E002	Henry	32	Alabama	Dpt2	HR
E003	Tom	22	Texas	Dpt3	Finance

Decompose the above table into two tables:

<EmpDetails>

Emp_ID	Emp_Name	Emp_Age	Emp_Location
E001	Jacob	29	Alabama
E002	Henry	32	Alabama
E003	Tom	22	Texas

<DeptDetails>

Dept_ID	Dept_Name
Dpt1	Operations
Dpt2	HR
Dpt3	Finance

Now, you won't be able to join the above tables, since **Emp_ID** isn't part of the **DeptDetails** relation. Therefore, the above relation has lossy decomposition.

- Decomposition is said to be **lossless**, if it is possible to reconstruct the original table by using a natural join without any loss of information.
- Decomposition is said to be **lossy**, if it is impossible to reconstruct the original table from the decomposed tables without loss of information.

Lossless Decomposition in DBMS

- Lossless join decomposition is a decomposition of a relation R into relations R_1 , R_2 such that if we perform a natural join of relation R_1 and R_2 , it will return the original relation R . This is **effective in removing redundancy from databases while preserving the original data...**
- In other words by lossless decomposition, it becomes feasible to reconstruct the relation R from decomposed tables R_1 and R_2 by using Joins.
- In Lossless Decomposition, we select the common attribute and the criteria for selecting a common attribute is that the common attribute must be a candidate key or super key in either relation R_1 , R_2 , or both.
- Decomposition of a relation R into R_1 and R_2 is a lossless-join decomposition if at least one of the following functional dependencies are in F^+ (Closure of functional dependencies)

Following conditions must hold: To check for lossless join decomposition using Functional Dependency set.

- 1. On taking Union of Attributes of relation R1 and relation R2 must be equal to the attribute of relation R, i.e. the attribute of relation R must be either in relation R1 or in relation R2. i.e. on adding an attribute of both the table (excluding duplicate attributes) we should get the total attribute of R. In case, this condition fails then no need to check further as this is the first prerequisite of the **lossless join decomposition**
- $\text{Attribute}(R1) \cup \text{Attribute}(R2) = \text{Attribute}(R)$
- 2. On doing intersection of Attributes of relation R1 and relation R2 must not be NULL, i.e., at least there should be one common attribute in both the table based on which you join both the table, In case, this condition fails no need to check further as this is the second prerequisite of the **lossless join decomposition**.
- $\text{Attribute}(R1) \cap \text{Attribute}(R2) \neq \Phi$

- 3. The common attribute must be a key for at least one relation (R1 or R2). i.e. suppose an attribute "A" is common in both R1 and R2 then either A should be key in R1 or A should be key in R2. In case this condition fails no need to check further as this is the last prerequisite of the **lossless join decomposition**.
- Attribute (R1) $\cap$ Attribute (R2) $\rightarrow$ Attribute (R1) or
- Attribute (R1) $\cap$ Attribute (R2) $\rightarrow$ Attribute (R2)

Properties of Decomposition

- Decomposition Must be Lossless

Decomposition must always be lossless, which means the information must never get lost from a decomposed relation. This way, we get a guarantee that when joining the relations, the join would eventually lead to the same relation in the result as it was actually decomposed.

- Dependency Preservation

In the dependency preservation, at least one decomposed table must satisfy every dependency.

If a relation R is decomposed into relation R1 and R2, then the dependencies of R either must be a part of R1 or R2 or must be derivable from the combination of functional dependencies of R1 and R2.

- Lack of Data Redundancy

It is also commonly termed as a repetition of data/information. According to this property, decomposition must not suffer from data redundancy. When decomposition is careless, it may cause issues with the overall data in the database. When we perform normalization, we can easily achieve the property of lack of data redundancy.

DECOMPOSITION

- * Decomposition addresses the problem of redundancy.
- * A relation R is decomposed to R_1 & R_2 if the natural join of R_1 and R_2 gives original relation R . Then decomposition is lossless.
- * If the natural join R_1 and R_2 does not give the original relation R , then decomposition is lossy.
The effect of lossy decomposition is the presence of spurious tuple on joining R_1 and R_2 .

Two properties that the relational schema should ~~not~~ have after decomposition.

(i) Lossless Join Property
(Non additive property)

(ii) Dependency Preserving Property

(i) Lossless Property: No spurious tuples generated

(ii) Dependency Preserving Property:
Ensures all FDs are retained even after decomposition.

(i) is more critical & mandatory requirement while decomposing.

(ii) can however be overlooked.

Q. Check the decomposition is lossy or lossless

1. R (ABCD)

$\{AB \rightarrow C, C \rightarrow A, C \rightarrow D\}$, Decomposition (D) = $\{AB, ACD\}$

2. R (ABCDEF)

$\{A \rightarrow B, C \rightarrow DE, AC \rightarrow F\}$, D = $\{BE, ACDEF\}$

3. R (ABCDE)

$\{A \rightarrow BC, CD \rightarrow E, B \rightarrow D, E \rightarrow A\}$, D = $\{ABC, ACDE\}$

Solution for question 1:

$$r_1 = AB, r_2 = ACD$$

Then,

$$r_1 \cup r_2 = r$$

$$r_1 \cap r_2 = A \neq \phi$$

$$r_1 \cap r_2 = A,$$

To check **A** is **Super Key** of r_1 or r_2 take closer of **A**,

$$A^+ = A \neq r_1 \neq r_2.$$

So, it is **lossy decomposition**.

Solution for question 2:

$$\mathbf{r_1 = BE , r_2 = ACDEF}$$

Then,

$$\mathbf{r_1 \cup r_2 = R}$$

$$\mathbf{r_1 \cap r_2 = E \neq \phi}$$

$$\mathbf{r_1 \cap r_2 = E, \text{ is Super Key of } r_1 \text{ or } r_2, E^+ = E \neq R}$$

So, it is **lossy decomposition**.

Solution for question 3:

$$\mathbf{r_1 = ABC , r_2 = ACDE}$$

Then,

$$\mathbf{r_1 \cup r_2 = r}$$

$$\mathbf{r_1 \cap r_2 = AC \neq \phi}$$

$\mathbf{r_1 \cap r_2 = AC}$, is **Super Key** of $\mathbf{r_1}$ or $\mathbf{r_2}$,

$$\mathbf{(AC)^+ = ACBDE = r, AC \rightarrow ACDE}$$

and

We can say $\mathbf{AC \rightarrow ACB}$.

So, **AC** is **Super Key** of $\mathbf{r_1}$ (**only one check is enough**).

So, it is **lossless decomposition**.

We can check another one but not necessary.

AC $\rightarrow$ ACDE is also true.

So, here **AC** is Super Key of both $\mathbf{r_1}$ or $\mathbf{r_2}$.

Consider the table R with attributes A, B, and C. The functional dependencies that hold on R are: $A \rightarrow B$, $C \rightarrow AB$. Which of the following statements is/are true?

I. The decomposition of R into R1(C, A) and R2(A, B) is lossless.

II. The decomposition of R into R1(A, B) and R2(B, C) is lossy.

(a) Only I

(b) Only II

(c) Both I and II

(d) Neither I nor II

$$\text{i) } R_1(C, A) \cup R_2(A, B) = R(A, B, C)$$

$$\text{ii) } R_1(C, A) \cap R_2(A, B) = A (\neq \phi)$$

$$\text{iii) } R_1 \cap R_2 = A$$

$$(A)^+ = \begin{array}{cc} A & B \\ | & \uparrow \\ \hline & \end{array} \quad (As, A \longrightarrow B) = R_2$$

i. e., A is **SK** of R_2

So, **(I)** – **TRUE**

– **Decomposition** of R into $R_1(C, A)$ and $R_2(A, B)$ is **lossless**.

$$\text{i) } R_1(A, B) \cup R_2(B, C) = R(A, B, C)$$

$$\text{ii) } R_1(A, B) \cap R_2(B, C) = B (\neq \phi)$$

$$\text{iii) } R_1 \cap R_2 = B$$

$$(B)^+ = B$$

i. e., B is neither **SK** of R_1 nor R_2

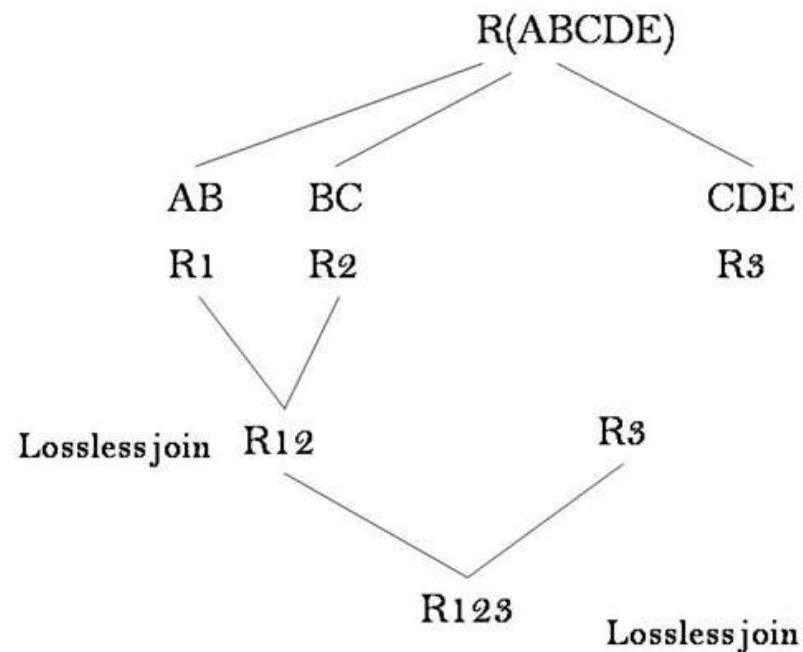
Hence, **(II) TRUE**

– as, decomposition of R into $R_1(A, B)$ and $R_2(B, C)$ is **lossy**.

$\therefore$ Both **I** and **II** are TRUE

$\therefore$ Answer: Option **(c)**.

As if now we are checking **lossless or lossy** for only **two relations**. But now we are going to check for more than two relations like we want to decomposed relation **R (ABCDE)** to **{AB, BC, CDE}**



At a time **three relations** can't check for lossless or lossy.

Between **three decompose relations** (R1, R2, R3), we check any two which can join by the **lossless way**, join them, after that again check resultant relation and rest of relation which is possible join with lossless way if, possible join them, if we got all combination of relation pair which make lossless join then the whole decomposition is lossless, if any of pair of relationships fails to join as lossless in this process then it is lossy.

We will not try **all possible combinations**, we only try **one possible pair (relation) way** and we try to select relation pair which **join as lossless**.

Example 2:

Let relation R (ABCDEG) and FDs $\{AB \rightarrow C, AC \rightarrow B, AD \rightarrow E, B \rightarrow D, BC \rightarrow A, E \rightarrow G\}$

Decomposition $D = \{ABC, ACDE, ADG\}$. Find it is lossy or lossless decomposition.

Solution:

Let

$$r_1 = ABC, r_2 = ACDE, r_3 = ADG$$

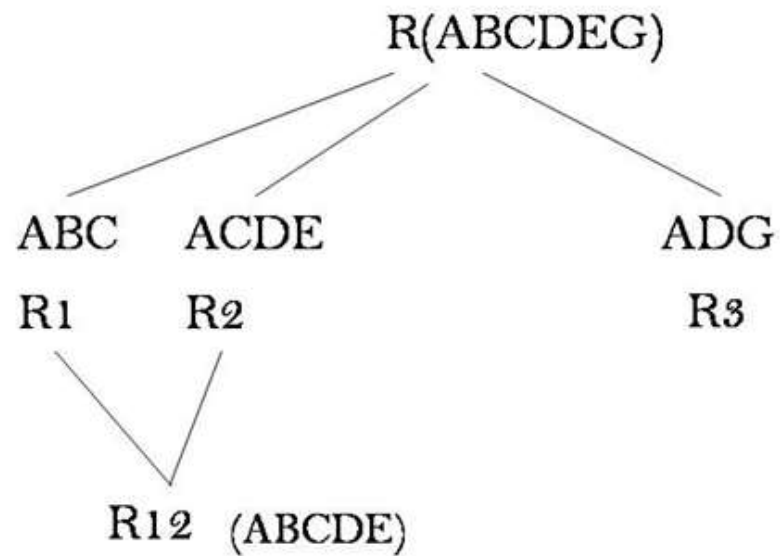
Looking for the first pair which is lossless

a. $r_1 \cap r_2 = AC, (AC)^+ = ACBDEG$

So, $AC \rightarrow ACBDEG$ then $AC \rightarrow ABC$

So, $r_1 \cap r_2 = AC$ is **super key** of r_1 table.

So, it is a **lossless join**. Take this pair and join them.

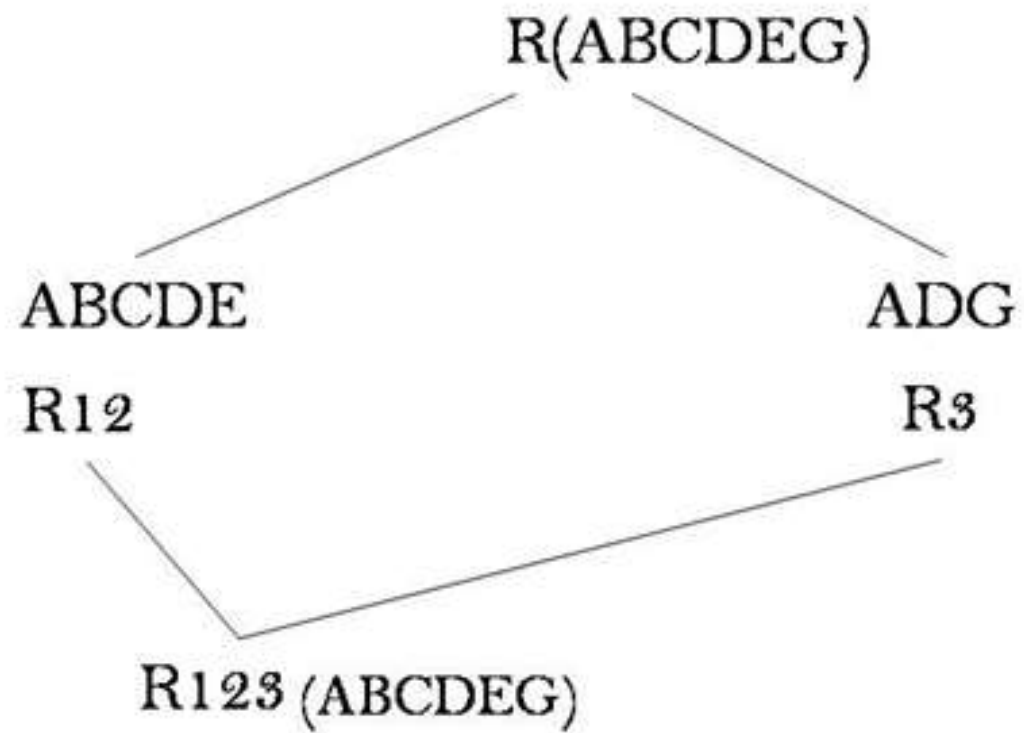


Now **two relations left** {ABCDE, ADG}

Checking for lossless

$\mathbf{r_{12} \cap r_3 = AD, (AD)^+ = ADEG}$ is **Super Key** if $\mathbf{r_3}$.

So, it is **lossless**, **join them**.



So, **total decomposition** is **lossless**.

Examples of Lossless Join Decomposition

Example 1:

Relation R (ABCDEFG) and **functional dependencies** $\{AB \rightarrow C, AC \rightarrow B, AD \rightarrow E, B \rightarrow D, BC \rightarrow A, E \rightarrow G\}$

Decomposition (D) = {AB, BC, ABDE, EG}

Solution:

Let

$$\mathbf{r_1 = AB, r_2 = BC, r_3 = ABDE, r_4 = EG}$$

Let try a search for the first pair of relation which makes lossless join decomposition.

a. $\mathbf{r_1 \cap r_2 = B,}$

$\mathbf{B^+ = BD}$ is **Super Key** of $\mathbf{r_1}$ or $\mathbf{r_2}$ – pair not possible.

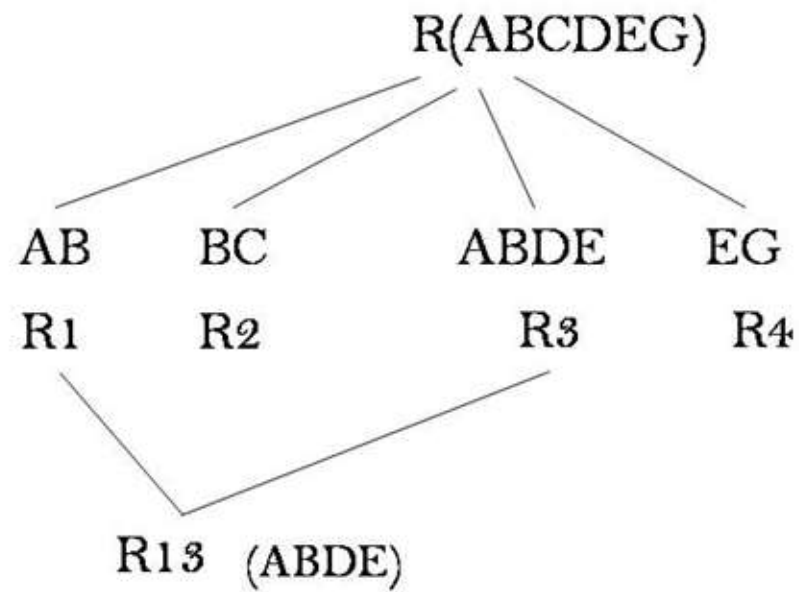
b. $\mathbf{r_1 \cap r_3 = AB, AB^+ = ABCDEG}$

We can say $\mathbf{AB \rightarrow ABCDEG}$ then $\mathbf{AB \rightarrow AB}$

So, $\mathbf{r_1 \cap r_3 = r_s}$ is **Super Key** of $\mathbf{r_1}$

So, it is **lossless**.

We can join them



Now we have **three relations**

$\{ABDE, BC, EG\}$

$r_{13} = ABDE, r_2 = BC, r_4 = EG$

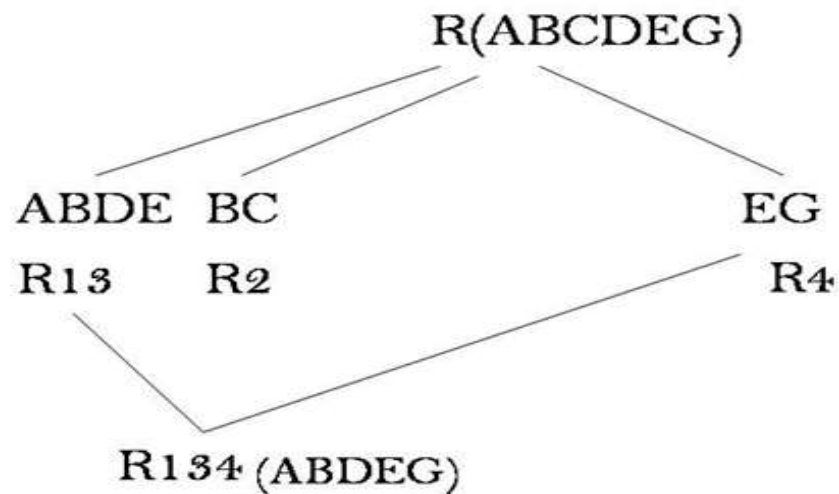
Check for joining pair

a. $r_{13} \cap r_2 = B, B^+ = BD$ is **Super Key** of r_{13} or r_2

then r_{13} and r_2 is **not possible**, it is **lossy**.

b. $r_{13} \cap r_4 = E, E^+ = EG$ is Super Key of r_4 ,

then it is **lossless** we can join them.



Now **two relations left** {ABDEG, BC}

$r_{134} = ABDEG, r_2 = BC$

$r_{134} \cap r_2 = B, B^+ = BD$ is **not Super Key** of r_{134} or r_2 ,

then **lossless join is not possible**.

So, the overall decomposition of **R (ABCDEG)** is **lossy decomposition**.

Dependency Preservation

- [Database Management System | Dependency Preserving Decomposition - GeeksforGeeks](#)

[Dependency preservation | PPT \(slideshare.net\)](#)

Dependency Preservation

- $(F_1 \cup F_2 \cup \dots \cup F_m)^+ = F^+$

Consider a relation R

$R \rightarrow F \{ \dots \text{with some functional dependency (FD)} \dots \}$

R is decomposed or divided into R1 with FD { f1 } and R2 with { f2 }, then

there can be three cases:

$f_1 \cup f_2 = F$ -----> Decomposition is dependency preserving.

$f_1 \cup f_2$ is a subset of F -----> Not Dependency preserving.

$f_1 \cup f_2$ is a super set of F -----> This case is not possible.

Points to remember

- 1) Write down all the non-trivial dependencies present in the child table.
- 2) Check the dependency hold true or not
 - a) Dependency can be directly present
 - b) Else take the closure of the determinant w.r.t FD given in the question and check the possibility
- 3) Take union of all possible FD.
- 4) Check all the original dependencies are preserved or not w.r.t possible FD.
 - a) Dependency can be directly present
 - b) Else take closure of the determinant w.r.t derived possible FD.

$R(W \ x \ y \ z)$

FD = $W \rightarrow x, x \rightarrow y, y \rightarrow z, z \rightarrow x$.

Decomposition = $R_1(Wx), R_2(x, y), R_3(z, z)$

R_1	R_2	R_3
<u>Wx</u>	<u>xy</u>	<u>xz</u>
$W \rightarrow x \checkmark$	$x \rightarrow y \checkmark$	$x \rightarrow z \checkmark$
$x \rightarrow W \ x$	$y \rightarrow x \checkmark$	$z \rightarrow x \checkmark$
$x^+ = xy.z$	$y^+ = yzx$	$x^+ = xyz$

Possible FD

$\{ \underset{\checkmark}{W \rightarrow x}, \underset{\checkmark}{x \rightarrow y}, y \rightarrow x, x \rightarrow z, \underset{\checkmark}{z \rightarrow x} \}$

$\{ W \rightarrow x, x \rightarrow y, y \rightarrow z, z \rightarrow x \} \rightarrow$ Depends

find y^+ ~~if~~ write possible FD.

Preserve

$R(ABCD)$

FD = $AB \rightarrow C$, $C \rightarrow D$, $D \rightarrow A$

Decomposition $R_1(ABC)$, $R_2(CD)$.

ABC

$AB \rightarrow C$ ✓

$BC \rightarrow A$ ✗

$CA \rightarrow B$ ✗

$A \rightarrow B$ ✗

$A \rightarrow C$ ✗

$B \rightarrow C$ ✗

$B \rightarrow A$ ✗

$C \rightarrow A$ ✓

$C \rightarrow B$ ✗

$BC^+ = BC$

$CA^+ = CA$

$C^+ = C \cancel{A}$

CD

$C \rightarrow D$ ✓

$D \rightarrow C$ ✗

$D^+ = DA$

possible FD = $AB \rightarrow C$, $C \rightarrow A$, $C \rightarrow D$.

$D^+ = D$

$D \rightarrow A$ Not

Preserved

Q. Consider the relation $R(W, X, Y, Z)$ with functional Dependency set

$F = \{$
 $WX \rightarrow Y$
 $Y \rightarrow W$
 $Y \rightarrow Z$
 $\}$

The relation is divided into two sub-relations $R_1(W, Y, Z)$ and $R_2(X, Y)$.

Is it dependency preserving decomposition?

Q. Check whether the decomposition is dependency preserving for the given relation $R(A,B,C,D,E)$ with the below FD Set and decomposition as $R_1(A,B,C)$, $R_2(B,C,D,E)$ and $R_3(A,E)$

$F = \{$

$A \rightarrow B$

$A \rightarrow C$

$DE \rightarrow C$

$DE \rightarrow B$

$C \rightarrow D$

- Let $R(A, B, C, D)$ be a relational schema with the following functional dependencies: $A \rightarrow B, B \rightarrow C, C \rightarrow D, D \rightarrow B$

The decomposition of R into $(A, B), (B, C), (B, D)$

- **(A)** gives a lossless join, and is dependency preserving
- (B)** gives a lossless join, but is not dependency preserving
- **(C)** does not give a lossless join, but is dependency preserving
- **(D)** does not give a lossless join and is not dependency preserving

Let $R(X,Y,Z,W)$ be a relational schema with the following functional dependencies:

$$X \rightarrow Y, Y \rightarrow Z, Z \rightarrow W \text{ and } W \rightarrow Y$$

The decomposition of R into $(X,Y),(Y,Z),(Y,W)$ is

- a. Lossy join, but is dependency preserving
- b. Lossless join, but is not dependency preserving
- c. Lossless join and dependency preserving
- d. Lossy join and not dependency preserving

Consider the relation $R(X, Y, W, Z, P, Q)$, and the set $F = \{XY \rightarrow W, XW \rightarrow P, PQ \rightarrow Z, XY \rightarrow Q\}$. R is decomposed into $R_1(Z, P, Q)$ and $R_2(X, Y, W, P, Q)$. Which of the following is true about R ?

1. Only lossless decomposition
2. Only dependency preserving decomposition
3. Neither lossless decomposition nor dependency preserving decomposition
4. Both lossless decomposition and dependency preserving decomposition

Consider a schema $R(X, Y, Z, W)$ and functional dependencies $X \rightarrow Y$ and $W \rightarrow Z$. Then the decomposition of R into $T1(X, Y)$ and $T2(Z, W)$ is

1. Dependency preserving and lossless join
2. Dependency preserving but not lossless join
3. Not dependency preserving but lossless join
4. Neither dependency preserving nor lossless join

Consider Relational schema R (ABCDEFGHIJ) and Functional dependency set

$S = \{AB \rightarrow C, A \rightarrow DE, B \rightarrow F, F \rightarrow GH, D \rightarrow IJ\}$

Which of following is Lossless and Dependency Preserving Decomposition

1. $D1 = \{ (DIJ), (ACE), (FGH), (BF), (ADC) \}$

2. $D2 = \{ (FGH), (DIJ), (ABDEF), (ABC) \}$

3. Both D1 and D2

4. None of these

(5)

How do we gauge if decomposition is good, given that dependencies cannot be preserved in most of the decompositions.

Remember, more critical requirement is lossless decomposition and dependency preservation property can however be overlooked.

lossless decomposition	Dependency Preserving	Decomposition
✓	✓	Good
✓	✗	Good
✗	✗	Bad
✗	✓	Bad

Normalization

- Assumption:

- A set of functional dependencies is given for each relation,
- Each relation has a designated primary key;
- This information combined with the tests (conditions) for normal forms drives the *normalization process* for relational schema design

- **Normalization of Relations**

- First proposed by Edgar Codd (1972)
- A database design technique that reduces data redundancy and eliminates undesirable characteristics like Insertion, Update and Deletion Anomalies.
- Normalization rules divides larger tables into smaller tables and links them using relationships.
- Purpose of Normalization in SQL is to eliminate redundant (repetitive) data and ensure data is stored logically.
- This process takes a relation schema through a series of tests to *certify* whether it satisfies a certain **normal form**
- Evaluating each relation against the criteria for normal forms and decomposing relations as necessary, can thus be considered as *relational design by analysis*
- Initially, Codd proposed three normal forms, which he called **first, second, and third normal form**
- Later he joined Raymond F. Boyce to develop the theory of **Boyce-Codd Normal Form**.

NORMALIZATION

- **Normalization** is the process of minimizing **redundancy** from a relation or set of relations.
- **Redundancy** in relation may cause **insertion, deletion and update anomalies**. So, it helps to minimize the redundancy in relations.
- **Normal forms** are used to eliminate or reduce redundancy in database tables.

1NF---- atomic attribute/ functional dependency

2NF----- partial dependency

3NF-----Transitive dependency

BCNF(Boyce Codd Normal Form)----special of 3NF

4NF----- multivalued dependency

5NF----- Join dependency

Normalization is the process of splitting relations into well structured relations that allow users to insert, delete, and update tuples/rows/records efficiently.

Without normalization many problems can occur when trying to load an integrated conceptual model (undecomposed relations) into the DBMS.

These problems arise from relations that are generated directly from user views, and are called anomalies. There are three types of anomalies: update, deletion and insertion anomalies.

Anomalies

- 1. Insertion**
- 2. Deletion**
- 3. Update**

1. Insertion Anomaly

An insertion anomaly is the inability to add data to the database due to absence of other data.

For example, assume if a new employee is hired but not immediately assigned to any dept then this employee could not be entered into the database or insert with null value in dept column. This results in database' inconsistencies .

Emp_dept(eno, ename, sal, gender, dname, dloc)

1, A, 20k, M ,CSE, GF

2, D, 30K, M, null, null

3, B, 40K, F, CSE, GF

4, C, 40K, F, CSE, GF

Insertion Anomaly solution

Emp_dept(eno, ename, sal, gender, dname, dloc)

1, A, 20k, M ,CSE, GF

2, D, 30K, M, null, null

3, B, 40K, F, CSE, GF

4, C, 40K, F, CSE, GF

Emp(eno, ename, sal, gender, dname)

1, A, 20k, M ,CSE

2, D, 30K, M, null **(No. of null values can be considerably reduced)**

3, B, 40K, F, CSE

4, C, 40K, F, CSE

Dept(dname, dloc)

1. CSE GF

2. PHY FF

2. Deletion Anomaly

A deletion anomaly is the unintended loss of data due to deletion of other data.

For example, if the emp B resigns and the record is deleted from the table below, then employee B and Dept PHY would cease to exist (because B is the only employee of PHY).

This results in database inconsistencies and is an example of how combining information that does not really belong together into one table can cause problems.

Emp_dept(eno, ename, sal, gender, dname, dloc)

1, A, 20k, M ,CSE, GF

2, B, 30K, M, PHY, TF

3, C, 40K, F, CSE, GF

4, D, 40K, F, CSE, GF

Deletion Anomaly Solution

Emp_dept(eno, ename, sal, gender, dname, dloc)

1, A, 20k, M ,CSE, GF

2, B, 30K, M, phy, FF

3, C, 40K, F, CSE, GF

4, D, 40K, F, CSE, GF

Emp(eno, ename, sal, gender, dname)

1, A, 20k, M ,CSE

2, B, 30K, M, phy (even if removed phy exists in dept table)

3, C, 40K, F, CSE

4, D, 40K, F, CSE

Dept(dname, dloc)

1. cse GF

2. phy ff

3. Update Anomaly

An update anomaly is a data inconsistency that results from data redundancy and a partial update.

Update anomalies occur when the same data is repeated in multiple rows, and changes are made in some but not all instances. For example, if an employee's address changes and the update is made in one row but not in others, the database will contain inconsistent data.

For example, the location of CSE in multiple places have different values(inconsistent). Therefore it is better to decompose the table

Emp_dept(eno, ename, sal, gender, dname, dloc)

1, A, 20k, M ,CSE, FF

2, B, 30K, M, PHY, TF

3, C, 40K, F, CSE, FF

4, D, 40K, F, CSE, GF

Update Anomaly solution

Emp_dept(eno, ename, sal, gender, dname, dloc)

1, A, 20k, M ,CSE, GF

2, A, 30K, M, phy, FF

3, B, 40K, F, CSE, GF

4, C, 40K, F, CSE, GF

Emp(eno, ename, sal, gender, dname)

1, A, 20k, M ,CSE

2, A, 30K, M, PHY

3, B, 40K, F, CSE

4, C, 40K, F, CSE

Dept(dname, dloc) (loc of the dept is stored only in this table)

1. CSE GF

2. PHY FF

Update Anomaly

Acc_Cust

Accno	Bal	Custid	Custname	city
A1	10k	C1	A	Mys
A2	35k	C2	B	Mys
A3	56k	C1	A	Blr
A4	78k	C3	C	Mlr
A5	34k	C4	D	Mys

Cust c1 blr– mys

Update acc_cust set city='mys' where accno='A1';

Accno	Bal	Custid
A1	10k	C1
A2	35k	C2
A3	56k	C1
A4	78k	C3
A5	34k	C4

Cust c1 blr– mys

Custid	Custname	city
C1	A	Mys
C2	B	Mys
C3	C	Mlr
C4	D	Mys

Update cust set city='mys' where custid='c1';

Why do we need Normalization?

- The main reason for normalizing the relations is removing these anomalies. Failure to eliminate anomalies leads to data redundancy and can cause data integrity and other problems as the database grows. Normalization consists of a series of guidelines that helps to guide you in creating a good database structure.

- **Data modification anomalies can be categorized into three types:**
- **Insertion Anomaly:** Insertion Anomaly refers to when one cannot insert a new tuple into a relationship due to lack of data.
- **Deletion Anomaly:** The delete anomaly refers to the situation where the deletion of data results in the unintended loss of some other important data.
- **Updation Anomaly:** The update anomaly is when an update of a single data value requires multiple rows of data to be updated.

- Types of Normal Forms:
- Normalization works through a series of stages called Normal forms. The normal forms apply to individual relations. The relation is said to be in particular normal form if it satisfies constraints.
- **Following are the various types of Normal forms:**

	1NF	2NF	3NF	4NF	5NF
Decomposition of Relation	R	R ₁₁ R ₁₂	R ₂₁ R ₂₂ R ₂₃	R ₃₁ R ₃₂ R ₃₃ R ₃₄	R ₄₁ R ₄₂ R ₄₃ R ₄₄ R ₄₅
Conditions	Eliminate Repeating Groups	Eliminate Partial Functional Dependency	Eliminate Transitive Dependency	Eliminate Multi-values Dependency	Eliminate Join Dependency

First Normal Form (1 NF)

Guideline : All the attributes should be atomic in nature.

Atomic attribute = single valued attribute

Emp(eno, ename, sal, phno) phno=mva

Is Emp in 1NF??? NO

Emp(eno, ename, sal, phno)

1, A, 10, 100

1, A, 10, 101

2, B, 20, 102

Emp(eno, ename, sal, phno1, phno2)

Emp(eno, ename, sal) & emp_ph(eno, phno)

1, 100

1, 101

First Normal Form (1NF)

Guideline: All attributes have to be atomic in nature
atomic = single valued.

Example: Consider a relation Emp with attributes eno, ename, sal and phone number. An employee can have more than 1 phone number.

⇒

eno	ename	sal	phNo
1	A	20K	100, 101
2	B	30K	102
3	C	30K	103, 104

Violating 1NF

⇒ Emp is not in 1NF so Normalise.

Soln1:

eno	ename	sal	phNo
1	A	20K	100
1	A	20K	101
2	B	30K	102
3	C	30K	103
3	C	30K	104

Satisfies 1NF but
lot of data
redundancy.
ename & sal
repeated.

Violates P.K rules
(eno is repeated)

Soln 2:

eno	ename	sal	ph1	ph2
1	A	20k	100	101
2	B	30k	102	NULL
3	C	30k	103	104

Satisfies 1NF
but generates
lot of null
values.

Soln 3:

Decompose Emp Table To Emp & Emp-ph.

Emp		
eno	ename	sal
1	A	20k
2	B	30k
3	C	30k

Emp-ph	
eno	ph
1	100
1	101
2	102
3	103
3	104

The above solution (soln 3) is the
ideal solution and
both Emp and emp-ph satisfies 1NF
as there is no mva.

Second Normal Form (2NF)

For a table to be in the Second Normal Form, it must satisfy two conditions:

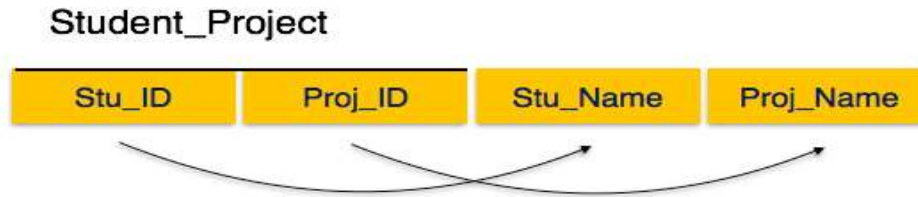
- The table should be in the First Normal Form.
- There should be no Partial Dependency.

Second Normal Form (2NF)

- Before we learn about the second normal form, we need to understand the following –
- **Prime attribute** – An attribute, which is a part of the candidate-key, is known as a prime attribute.
- **Non-prime attribute** – An attribute, which is not a part of the prime-key, is said to be a non-prime attribute.
- If we follow second normal form, then every non-prime attribute should be fully functionally dependent on prime key attribute. That is, if $X \rightarrow A$ holds, then there should not be any proper subset Y of X , for which $Y \rightarrow A$ also holds true.

Second Normal Form (2NF)

- To check if a relation is in 2nd normal form (2NF), we need to verify if it is already in 1st normal form (1NF) and if no non-prime attribute (an attribute not part of the candidate key) is partially dependent on any subset of the candidate key; meaning, every non-prime attribute must depend on the entire candidate key, not just a part of it.



- We see here in Student_Project relation that the prime key attributes are Stu_ID and Proj_ID. According to the rule, non-key attributes, i.e. Stu_Name and Proj_Name must be dependent upon both and not on any of the prime key attribute individually. But we find that Stu_Name can be identified by Stu_ID and Proj_Name can be identified by Proj_ID independently. This is called **partial dependency**, which is not allowed in Second Normal Form.

Student

Stu_ID	Stu_Name	Proj_ID
--------	----------	---------

Project

Proj_ID	Proj_Name
---------	-----------

We broke the relation in two as depicted in the above picture. So there exists no partial dependency.

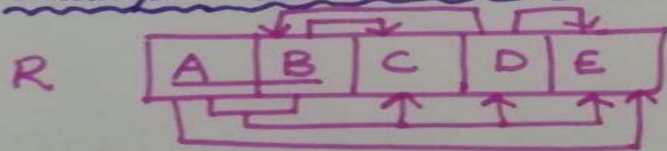
(3)

Second Normal Form (2NF)

Guidelines:

- (i) Should satisfy 1NF
- (ii) There should not be any partial dependency between key attribute and non key attribute. {primary key being in LHS (determinant) and non key attribute being in RHS (dependent)}.

Example to understand partial dependency



Given AB is the key attribute
CDE non key attribute

$AB \rightarrow CDE$ (Fully FD)

$B \rightarrow C$ (partial dependency)

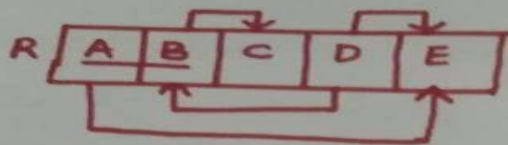
$D \rightarrow B$ (Functional dependency)

$A \rightarrow E$ (partial dependency)

$D \rightarrow E$ (functional dependency)

LHS is a
part of
key attribute

Example: Given R. Check if R is satisfying 2NF.



Soln. Given $A \rightarrow E$

$D \rightarrow B$

$B \rightarrow C$

$D \rightarrow E$

$AB \rightarrow CDE$ (Implicit: key attributes always determine non key attributes)

Step 1:

The above relation R is in 1NF because no multi valued attribute.

Step 2: Not in 2NF because of

PDs $A \rightarrow E$ and $B \rightarrow C$. So decompose R into R₁ and R₂.

$R_1(ABD)$ and $R_2(BCE)$

(OR)

$R_1(ABD)$ $R_2(BC)$ $R_3(AE)$

Here these relations satisfy 2NF

GREEN:

$R_1 \text{ int } R_2 = B$

$R_2 - R_1 = C, E$

$B \rightarrow C, E$

BLUE:

$R_1 \text{ int } R_2 = B$

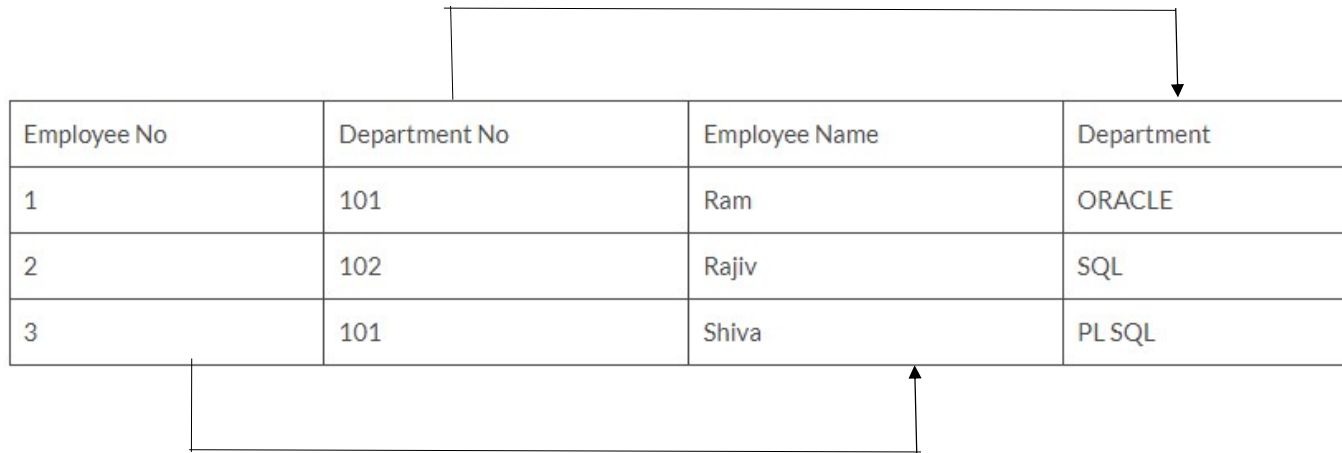
$R_2 - R_1 = C$

$B \rightarrow C$

$R_1 \text{ int } R_3 = A$

$R_3 - R_1 = E$

$A \rightarrow E$



Employee No	Department No	Employee Name	Department
1	101	Ram	ORACLE
2	102	Rajiv	SQL
3	101	Shiva	PLSQL

Here We will see that there is composite key as{ Employee No,Department No}

In above example we can see that department .Here We will see that there is composite key as{ Employee No,Department No},Employee No is dependent on Employee Name and Department is dependent on Department No.We can split the above table into 2 different tables:

Table 1:Employee_NO table

Employee No	Department No	Employee Name
1	101	Ram
2	102	Rajiv
3	101	Shiva

Table 2:Department table

Department No	Department
101	ORACLE
102	SQL

- Now we have simplified the table in to 2NF where each entity of table is functionally dependent on primary key.

Quick Recap

- For a table to be in the Second Normal form, it should be in the First Normal form and it should not have Partial Dependency.
- Partial Dependency exists, when for a composite primary key, any attribute in the table depends only on a part of the primary key and not on the complete primary key.
- To remove Partial dependency, we can divide the table, remove the attribute which is causing partial dependency, and move it to some other table where it fits in well.